ASSIGNMENT 1

Name: Yavanika S
Reg. No: 192521258
Slot A: CSA0304
Guided by: Dr.Manoj H.D

A) Define Data, Object and Information [5 Marks]

Definitions

1. Data: Data means raw, unprocessed facts such as numbers, text or symbols. On its own it has no meaning and cannot be used to take a decision. **Example:** 85, 90, 78 (marks with no context).

2. Object: An object is a real-world entity and the run-time instance of a class. It bundles its data (attributes) and the operations on that data (methods) into one unit, and it has its own identity. **Example:** Student s1 with name = "Ravi", marks = 85 and the method result().

3. Information: Information is data that has been processed and organised so that it becomes meaningful and useful for taking decisions. **Example:** Average = 84.33, which gives the result "Distinction".

From Data to Information

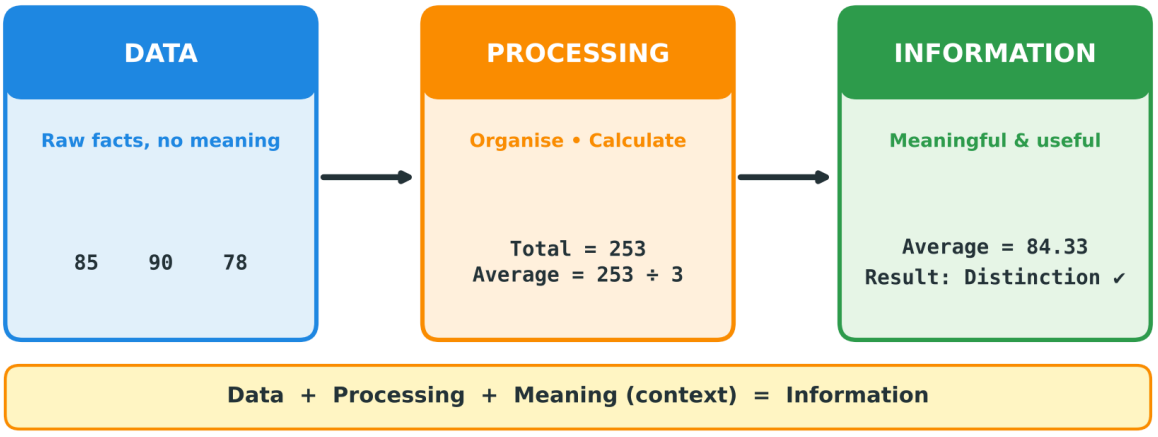


Figure 1: Data is processed to give Information

Raw data such as the marks 85, 90 and 78 is only a set of numbers. When it is processed (organised and calculated: total = 253, average = 253 ÷ 3), it gains context and meaning and becomes information. Hence **Data + Processing + Meaning = Information**. Data is the input to a system, and information is its output.

Data vs Information

- **Meaning:** Data is raw facts with no context; information is processed and meaningful.
- **Role:** Data is the input to a system; information is the output of a system.
- **Use:** Data alone cannot support decisions; information supports decisions.

Class and Object

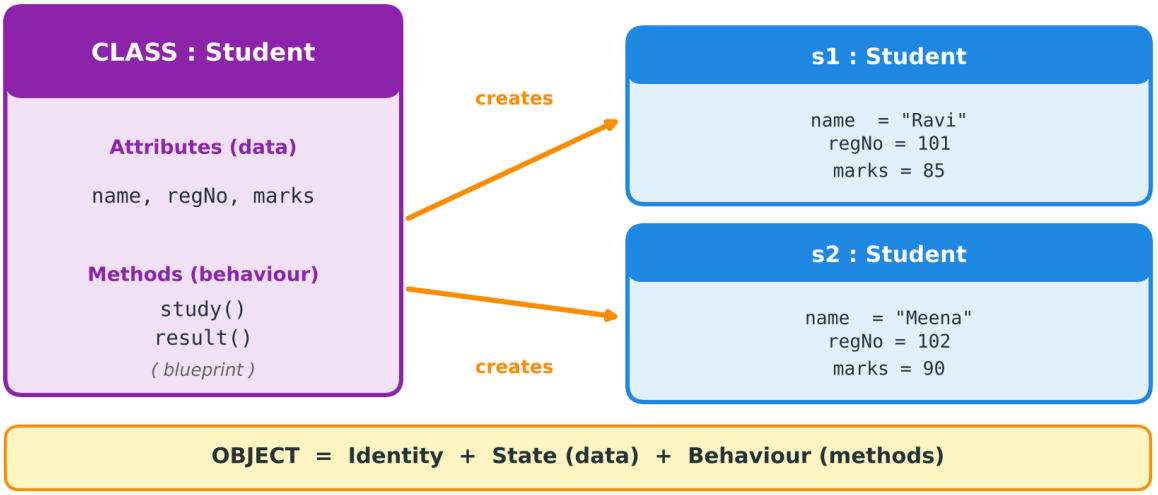


Figure 2: A class is the blueprint, objects are its instances

A **class** is a blueprint or template that defines the attributes and methods common to a group of similar entities. An **object** is a real instance created from that class. Here the class Student defines name, marks and result (); the objects s1 and s2 are created from it, each holding its own data values while sharing the same structure and behaviour. One class can therefore create any number of objects, and every object has its own identity.

B) C Program for Matrix Operations [10 Marks]

Aim

To write a menu-driven C program that performs **transpose, addition, subtraction and multiplication** of matrices using 2-D arrays and functions.

Algorithm

1. **START.**
2. Read the order and elements of matrix **A (r1 × c1)** and matrix **B (r2 × c2)**.
3. Display the menu and read **choice**.
4. **1 Transpose:** for every i, j set t[j][i] = a[i][j].
5. **2 Add:** if r1 = r2 and c1 = c2, set s[i][j] = a[i][j] + b[i][j]; otherwise print an error.
6. **3 Subtract:** same condition, set d[i][j] = a[i][j] − b[i][j].
7. **4 Multiply:** if c1 = r2, set p[i][j] = Σ a[i][k] × b[k][j] for k = 0 … c1−1; otherwise print an error.
8. Display the result and go back to step 3 until **choice = 5**.
9. **STOP.**

Rules to Remember

- **Transpose:** no condition (any r × c); formula t[j][i] = a[i][j]; result order c × r.
- **Addition:** r1 = r2 and c1 = c2; formula s[i][j] = a[i][j] + b[i][j]; result order r × c.
- **Subtraction:** r1 = r2 and c1 = c2; formula d[i][j] = a[i][j] − b[i][j]; result order r × c.
- **Multiplication:** c1 = r2; formula p[i][j] += a[i][k] × b[k][j]; result order r1 × c2.
- **Time complexity:** transpose, addition and subtraction take O(r × c); multiplication takes O(r1 × c1 × c2), i.e. O(n³) for n × n matrices.
- **Why a MAX × MAX array:** C needs a fixed size at compile time, so the program uses int m[10][10] and works with the first r rows and c columns.

Flowchart

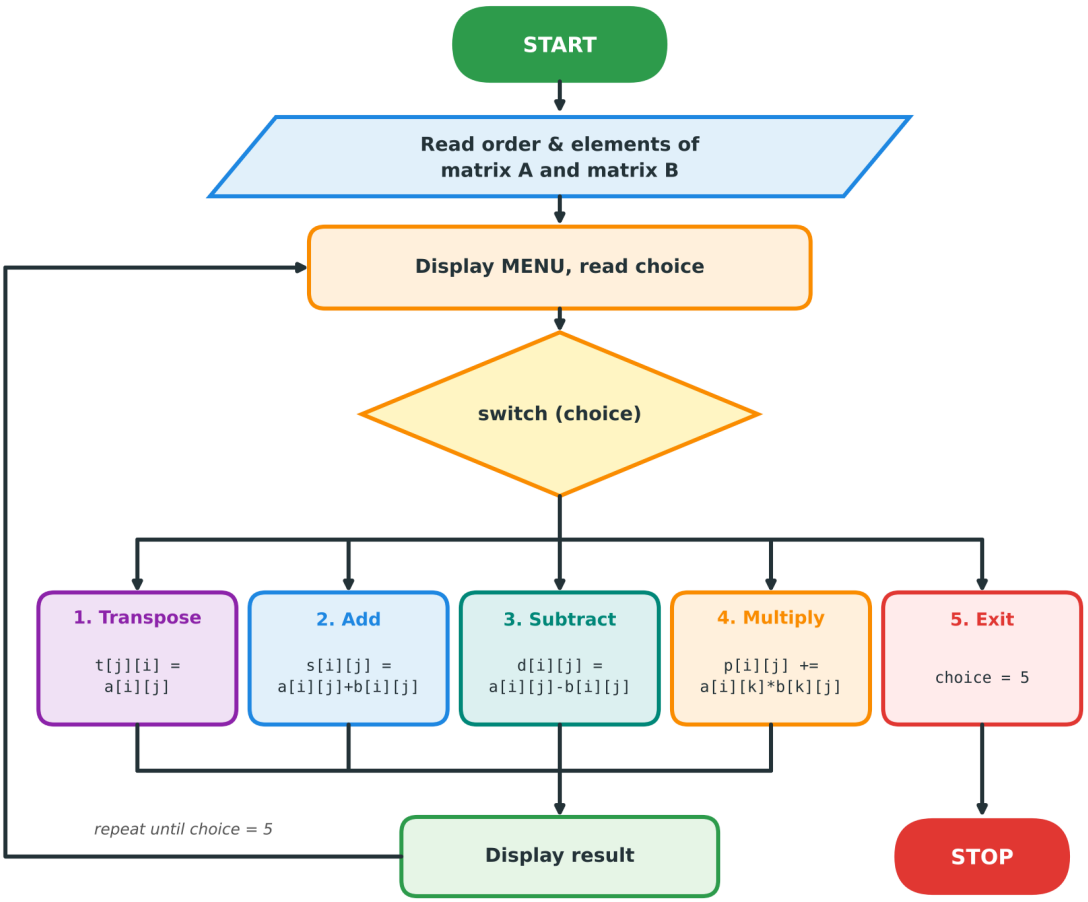


Figure 3: Flowchart of the menu-driven matrix program

The flowchart shows the logic of the program. After **START**, the orders and elements of both matrices are read and the menu is displayed. A **switch** on the user's choice sends control to one of five branches: transpose, add, subtract, multiply or exit. The add, subtract and multiply branches first test their condition; if it fails, an error message is printed. The result is displayed and control loops back to the menu until choice = 5, which leads to **STOP**.

Program

```
#include <stdio.h>
#define MAX 10

void read(int m[MAX][MAX], int r, int c, char n) {
    printf("Enter elements of matrix %c (%d x %d):\n", n, r, c);
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            scanf("%d", &m[i][j]);
}

void show(int m[MAX][MAX], int r, int c) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++)
            printf("%5d", m[i][j]);
        printf("\n");
    }
}

int main() {
    int A[MAX][MAX], B[MAX][MAX], R[MAX][MAX];
    int r1, c1, r2, c2, ch, i, j, k;

    printf("Enter rows and columns of matrix A: ");
    scanf("%d %d", &r1, &c1);
    read(A, r1, c1, 'A');
    printf("Enter rows and columns of matrix B: ");
    scanf("%d %d", &r2, &c2);
    read(B, r2, c2, 'B');

    do {
        printf("\n1.Transpose 2.Add 3.Subtract 4.Multiply 5.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1:
                for (i = 0; i < r1; i++)
                    for (j = 0; j < c1; j++)
                        R[j][i] = A[i][j];
                show(R, c1, r1);
                break;
            case 2:
            case 3:
                if (r1 != r2 || c1 != c2) { printf("Orders differ!\n"); break; }
                for (i = 0; i < r1; i++)
                    for (j = 0; j < c1; j++)
                        R[i][j] = A[i][j] + (ch == 2 ? B[i][j] : -B[i][j]);
                show(R, r1, c1);
                break;
            case 4:
                if (c1 != r2) { printf("Not possible: c1 != r2!\n"); break; }
                for (i = 0; i < r1; i++)
                    for (j = 0; j < c2; j++) {
                        R[i][j] = 0;
                        for (k = 0; k < c1; k++)
                            R[i][j] += A[i][k] * B[k][j];
                    }
                show(R, r1, c2);
                break;
            case 5: break;
            default: printf("Invalid choice!\n");
        }
    } while (ch != 5);
    return 0;
}
```

Sample Input and Output

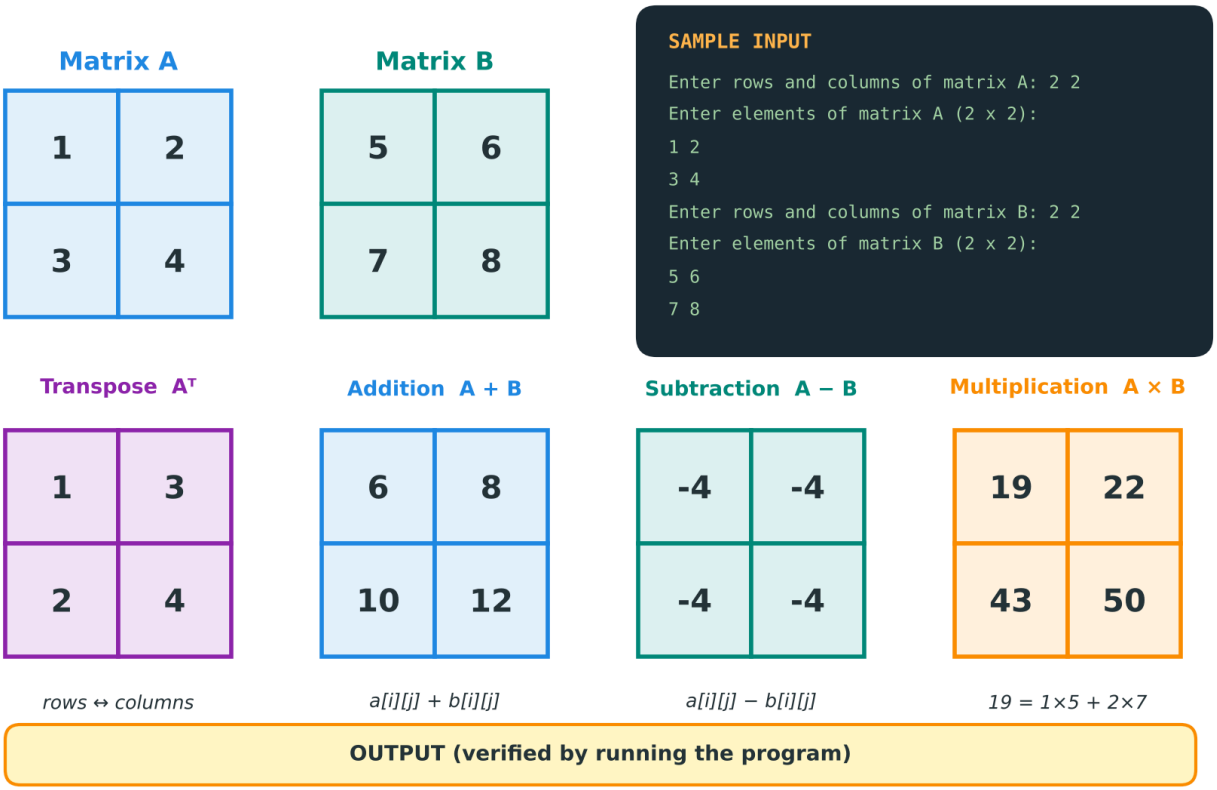


Figure 4: Sample input and the output of every menu option

For A = [1 2; 3 4] and B = [5 6; 7 8], the program gives: transpose of A = [1 3; 2 4]; A + B = [6 8; 10 12]; A − B = [−4 −4; −4 −4]; and A × B = [19 22; 43 50]. For example, 19 = (1 × 5) + (2 × 7). The output was verified by running the program.

C) Role of Arrays in Memory Management [5 Marks]

An array is a collection of elements of the **same data type** stored in **contiguous memory locations** under a single name. This simple layout is what makes memory use predictable and access fast.

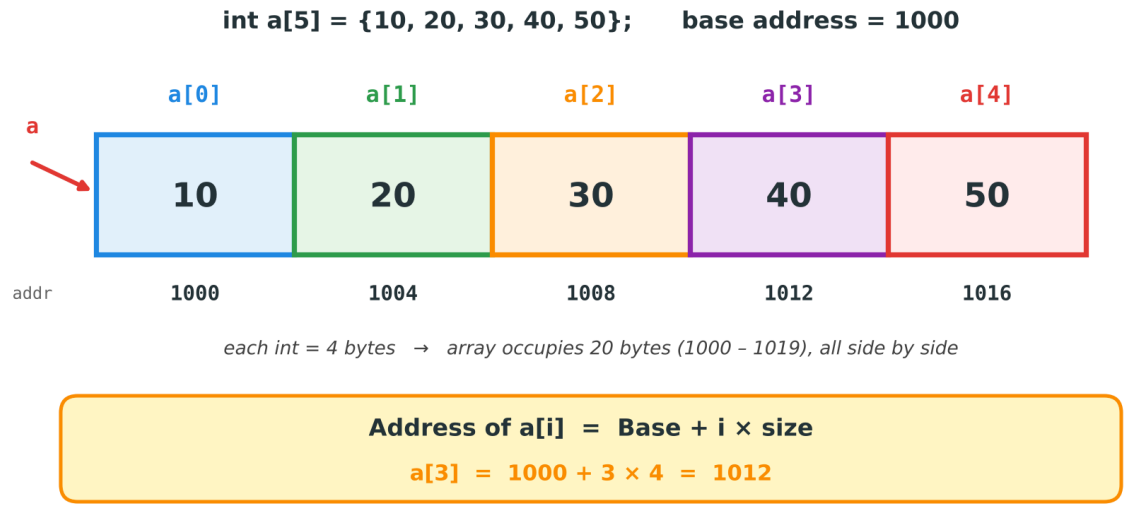


Figure 5: An int array stored in consecutive memory addresses

In the figure, the five elements of an int array occupy addresses 1000, 1004, 1008, 1012 and 1016, each element taking 4 bytes. Because the elements are consecutive, the address of any element is found directly by the formula **Address = Base + i × size**. For example, a[3] is at 1000 + 3 × 4 = 1012.

Roles of Arrays

1. **Contiguous allocation:** All elements sit side by side in one block, with no gaps and no extra pointers. Example: int a[5] needs 5 × 4 = 20 bytes.
2. **Fast direct access:** The address is calculated, not searched: Base + i × size. Access time is O(1). Example: a[3] = 1000 + 3 × 4 = 1012.
3. **Static allocation:** The size is fixed at compile time. Local arrays live on the stack, global and static arrays in the data segment, and the memory is released automatically. Example: int a[5];
4. **Dynamic allocation:** malloc() or calloc() create the array on the heap at run time, and free() returns the memory. Example: p = malloc(5 * sizeof(int));
5. **Cache friendly:** Neighbouring elements are loaded together in a cache line, so sequential loops run fast. Example: for (i = 0; i < 5; i++) sum += a[i];
6. **2-D arrays:** A matrix is stored row by row (row-major) as one linear block. Example: m[i][j] is at Base + (i × cols + j) × size.

Where Arrays Live in Memory

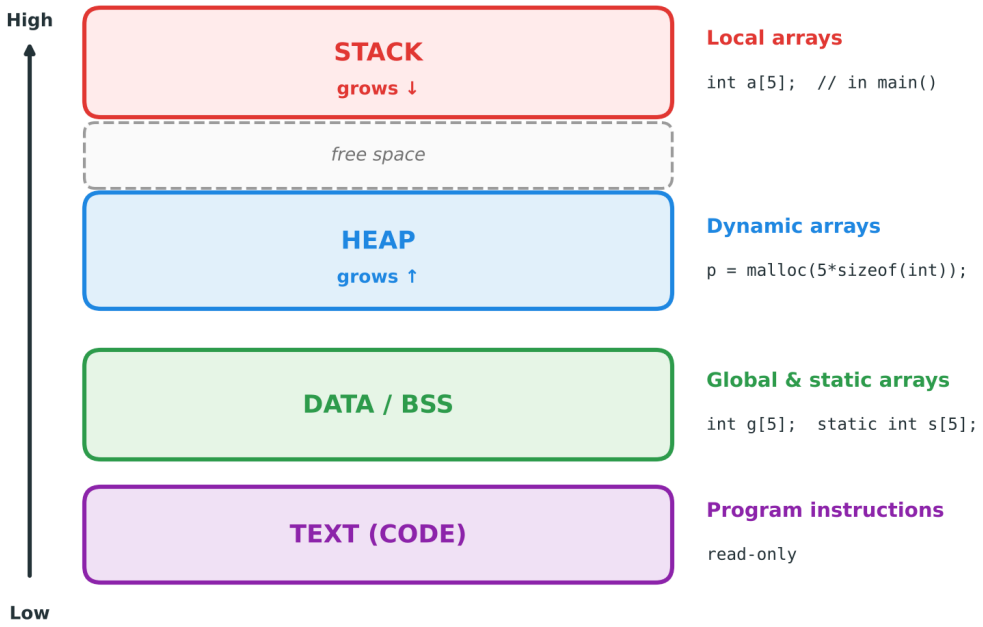


Figure 6: Arrays in the stack, heap and data segments of a program

A running program's memory is divided into the text (code), data, heap and stack segments, with free space between the heap and the stack. **Local arrays** are placed on the stack and are freed automatically when the function returns. **Global and static arrays** are placed in the data segment and last for the whole program. **Dynamic arrays** created by malloc() are placed on the heap and must be released by the programmer using free().

Advantages

- Simple to declare and use.
- Very fast index access, O(1).
- No extra memory for links or pointers.
- Cache friendly, so traversal is fast.

Limitations

- Fixed size, which leads to wasted memory or overflow.
- Insertion and deletion need shifting of elements.
- C does not check array bounds.
- Needs one large contiguous free block of memory.

Conclusion

Arrays store same-type elements in contiguous memory, which gives constant-time access through a simple address formula, good cache performance and predictable memory use. Their fixed size and

lack of bounds checking are the main drawbacks, which dynamic allocation and careful indexing help to manage.